

ARCHITECTURE & MEASURED METRICS REPORT

Sovereign Workbench

An air-gapped agentic AI workbench for refinery knowledge work, running entirely on one 6 GB laptop GPU. This report documents every component that exists today, every contract it honours, and every number it can actually defend.

Not an AI we *promise* is offline. An AI that *physically cannot* reach the internet, and shows you the blocked packets.

DEMO LEGS BUILT

3/8

Sovereignty, router, live model addition

TEST SUITE

36

passed, 1 strict xfail

ROUTER ACCURACY

75.8%

against a 90% gate — the one open failure

VRAM IN USE

0 GB

no model loads before leg 4

- 01

Executive summary & module inventory
- 02

System architecture
- 03

Containment — three independent layers
- 04

The red button — egress sequence
- 05

The audit chain & tamper evidence
- 06

The deterministic router
- 07

Router accuracy — the open gap
- 08

Error structure — the confusion matrix
- 09

Misroute inventory — all 16 errors
- 10

VRAM budget & model registry
- 11

Live model addition & acceptance gates
- 12

Roadmap & scaling profiles

What exists, and what it proves

The system is deliberately built depth-first: three of the eight demo legs are complete and hardened rather than eight sketched. Everything below was measured by running the code in this repository, not copied from the design document.

PYTHON + CONFIG

1,949

lines across 15 tracked files — small enough that a judge can read all of it

ROUTING DECISION

0.073ms

mean, full path incl. registry resolve · budget is 50 ms

LABELLED CORPUS

176

refinery prompts, 11 task types, 16 each

RUNTIME DEPS

4

fastapi · uvicorn · pydantic · pyyaml.
No CDN, no npm, no framework.

The three claims this build can defend

- 1 · Containment is demonstrable, not asserted.** A UI button attempts a real TCP connection to `api.openai.com:443`. It deliberately bypasses the application-layer guard so the *firewall* is what kills it — the destination IP and port then appear in the panel, read from the OS drop log.
- 2 · Every action is tamper-evident.** The audit log is an append-only hash chain whose digest covers the whole record, not just the payload. Editing one byte of any record breaks verification at that sequence number and orphans every record after it.
- 3 · Adding a model is a config edit.** `config/models.yaml` is the only file that names a model. A fitting model is accepted and routed to, an over-budget one is rejected with a visible reason, and malformed YAML leaves the previous registry serving — all with zero code change and zero restart.

The one claim it cannot yet defend

Router accuracy is 75.8% against a 90% gate. This is recorded in the test suite as a `strict xfail` rather than hidden — the day a better scorer lands, the suite fails until the xfail is deleted.

The cause is diagnosed, not mysterious. The lexical TF-IDF scorer has plateaued: leave-one-out CV on the training split alone gives 74.5% for class centroids and 64.5% for nearest-exemplar, and up-weighting the opening intent phrase made it worse at every setting tried. Section 9.2 of the charter originally specified *embedding* similarity, and it was right. Closing the gap needs a dense encoder staged on disk — see page 07.

Deliberate scope discipline

No LangChain, LlamaIndex, CrewAI or AutoGen — the charter forbids any framework that hides the agent loop, because the token budget has to stay controllable on 6 GB and judges read source. No multi-user auth, no multi-GPU, no fine-tuning, no cloud fallback behind any flag. The frontend is one 212-line HTML file with no build step. These are recorded non-goals, not omissions.

Module inventory — everything currently in the repository

FILE	LINES	RELATIVE SIZE	ROLE
core/classifier.py	224		TF-IDF class-centroid scorer, modality detection, hard overrides
web/index.html	212		The whole UI — 4 panels, vanilla JS, zero dependencies
config/routing_exemplars.jsonl	176		176 hand-labelled refinery prompts across 11 task types
tests/test_router.py	175		Accuracy gate (strict xfail), overrides, RouteDecision
core/registry.py	159		models.yaml load / validate / hot-reload, VRAM budget gate
core/audit.py	151		Hash-chained append-only JSONL + `audit verify` CLI
core/orchestrator.py	131		FastAPI app: 7 routes + 1 websocket, loopback bind
core/net_guard.py	122		socket.connect patch; the red button's bypass path
tests/test_registry.py	119		Accept / reject / malformed-YAML reload behaviour
core/router.py	107		RouteDecision assembly and the user-facing `reason` string
sovereignty/monitor.py	90		Incremental pfirewall.log tail → websocket
tests/test_audit.py	88		Chain integrity, tamper detection, resume-from-file
sovereignty/verify.ps1	69		Five pre-demo assertions; exits 0 only if contained
sovereignty/firewall.ps1	69		Outbound default-DENY + drop logging; shown verbatim in the UI
config/models.yaml	57		The only file that names a model

One process, four layers, no outbound edge

Every arrow in this diagram terminates inside the machine. There is no component whose job is to talk to a remote service, and no configuration flag that would create one.

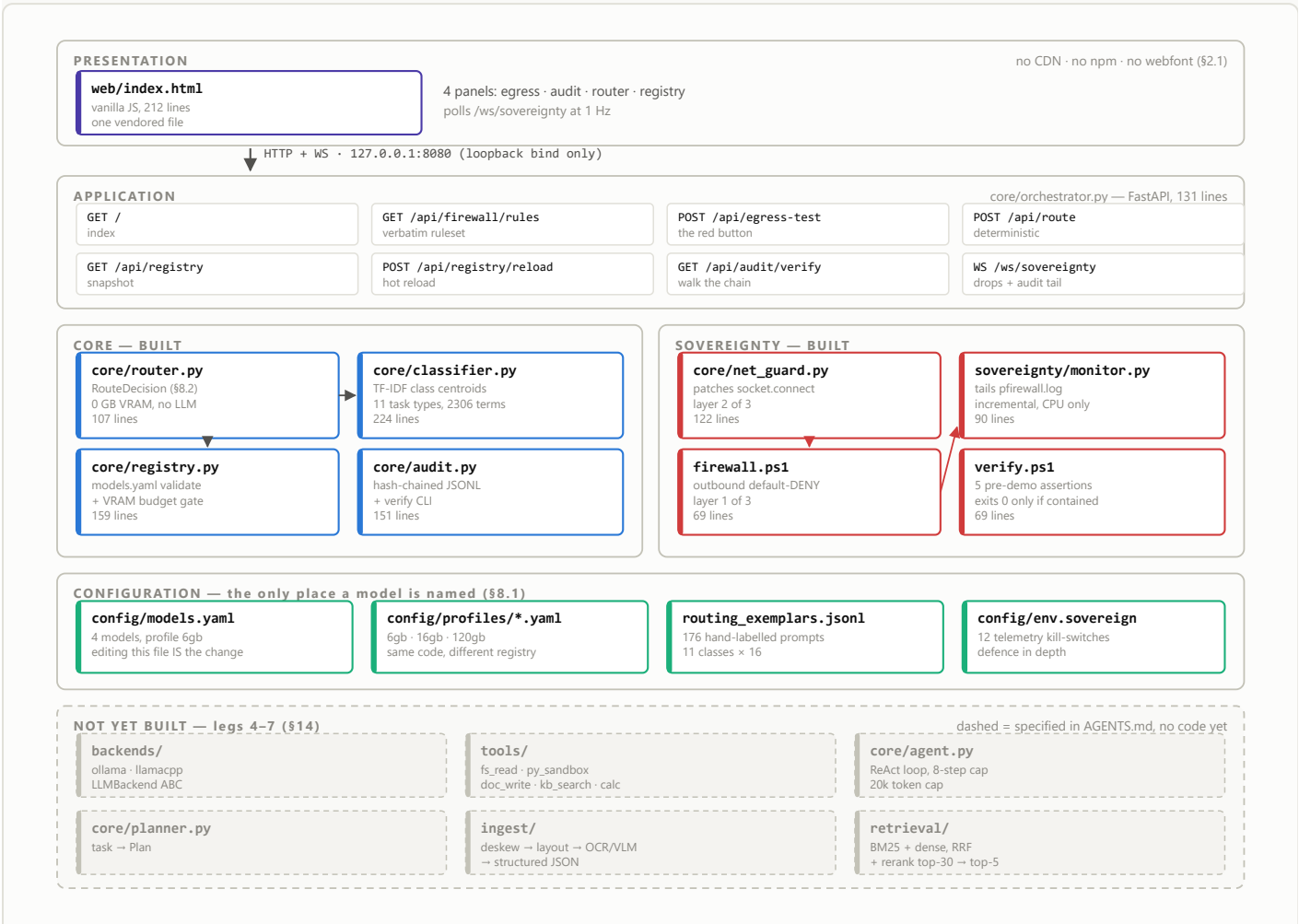


Figure 2.1 — Component architecture. Solid boxes are built and tested today; dashed boxes are specified in the charter with no code yet. The application layer is a single FastAPI process bound to 127.0.0.1:8080 — never 0.0.0.0 — serving seven HTTP routes and one websocket. Core and sovereignty modules are separated because they fail differently: a bug in core degrades a feature, a bug in sovereignty invalidates the entire thesis.

Request lifecycle

Application state lives on `app.state`, populated once in the lifespan handler: the audit log, the drop watcher, the registry, the router, and `resident` — which model Ollama currently holds. The registry singleton is the only permitted global mutable state in the codebase.

`install_guard()` patches `socket.connect` process-wide at startup, before any request is served. The first audit record of every session is written at that moment, so a session with no `session_start` record is a session that never booted cleanly.

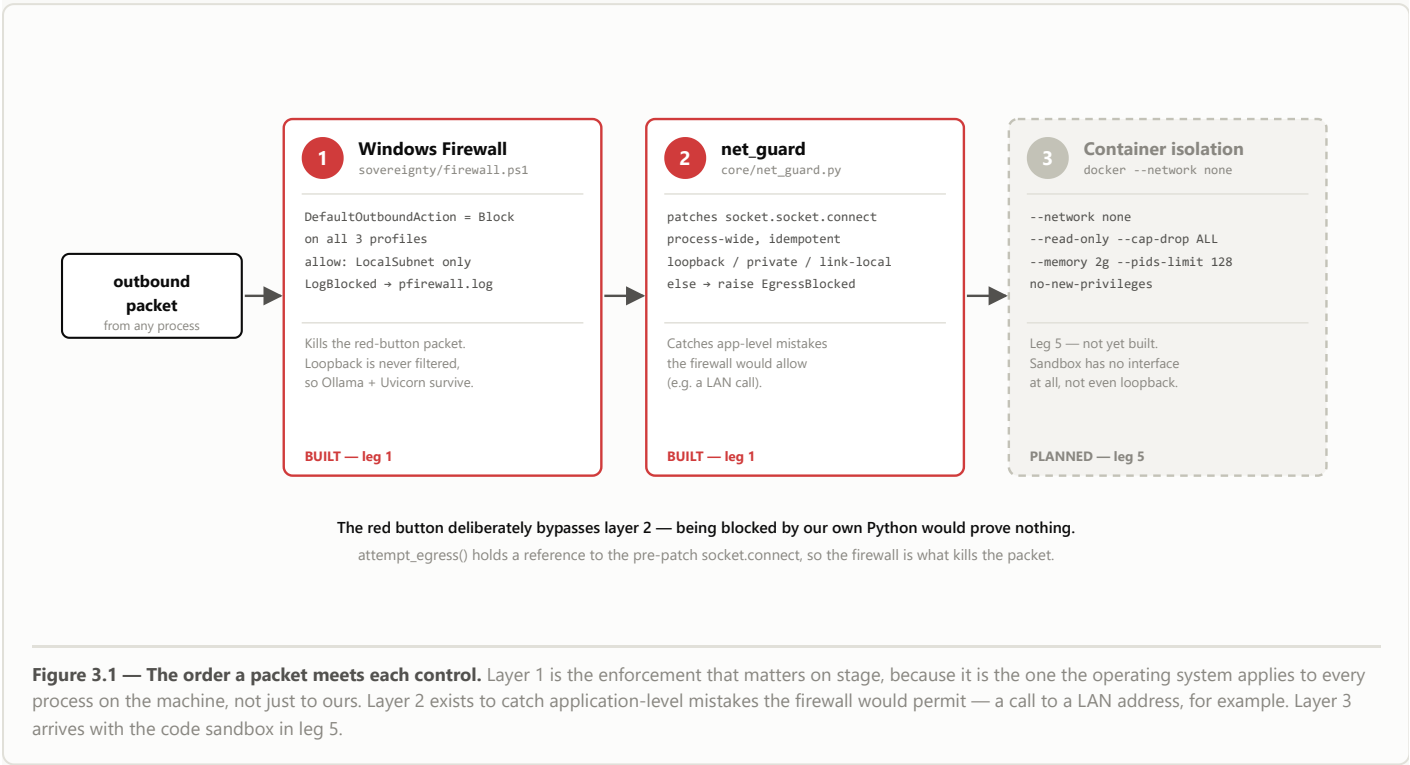
Why routing happens before any model loads

The router touches no weights and consults no LLM, so a decision costs 0.073 ms and 0 GB of VRAM. That is not a micro-optimisation — it is what makes the model swap presentable. On a 6 GB card exactly one LLM can be resident, so switching models costs 2–5 seconds even when weights are warm in the page cache.

Because the decision and its human-readable rationale are ready instantly, the UI renders them *while* Ollama is still loading. The delay stops reading as a stall and starts reading as an explanation.

Three independent containment layers

Defence in depth means no single misconfiguration restores internet access. The layers are independent by design: one is an OS control, one is a process control, one is a kernel namespace.



A fourth layer: configuration

config/env.sovereign disables telemetry at the library level for every dependency that has any — Hugging Face, Chroma, Gradio — plus the generic DO_NOT_TRACK. The firewall already blocks these calls; switching them off in config as well means a firewall misconfiguration is not a single point of failure.

```
# config/env.sovereign — 12 switches
OLLAMA_HOST=127.0.0.1:11434 ← loopback, never 0.0.0.0
OLLAMA_MAX_LOADED_MODELS=1 ← $4.2.1, single-tenant GPU
OLLAMA_KV_CACHE_TYPE=q8_0 ← halves KV memory
OLLAMA_FLASH_ATTENTION=1
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
HF_HUB_DISABLE_TELEMETRY=1
DO_NOT_TRACK=1
ANONYMIZED_TELEMETRY=False
GRADIO_ANALYTICS_ENABLED=False
```

Pre-demo verification — five assertions

sovereignty/verify.ps1 is run on stage and exits 0 only if every check passes. It is deliberately a separate script from the app: it checks the machine, not our opinion of the machine.

ASSERTION	FAILURE MEANS
Outbound is default-DENY on all 3 profiles	firewall not armed
Drop logging enabled, log file present	panel will read zero
Zero established external TCP sockets	something is still connected
Audit chain verifies end to end	log was tampered with
Ollama bound to loopback only	model server is reachable off-box

Honest failure mode. If the firewall is not armed, the red button reports !! CONTAINMENT FAILED rather than quietly succeeding. If the monitor cannot read pfirewall.log — it is Administrator-only by default — the panel says so in yellow rather than reporting zero drops. Both are the checks being honest, and both are worth pointing at during the demo.

What happens when the system tries to phone home

This is the single most memorable moment in the demo, so it is built to be unfakeable: a real socket, a real destination, a real four seconds of silence, and a drop record produced by the operating system rather than by our code.

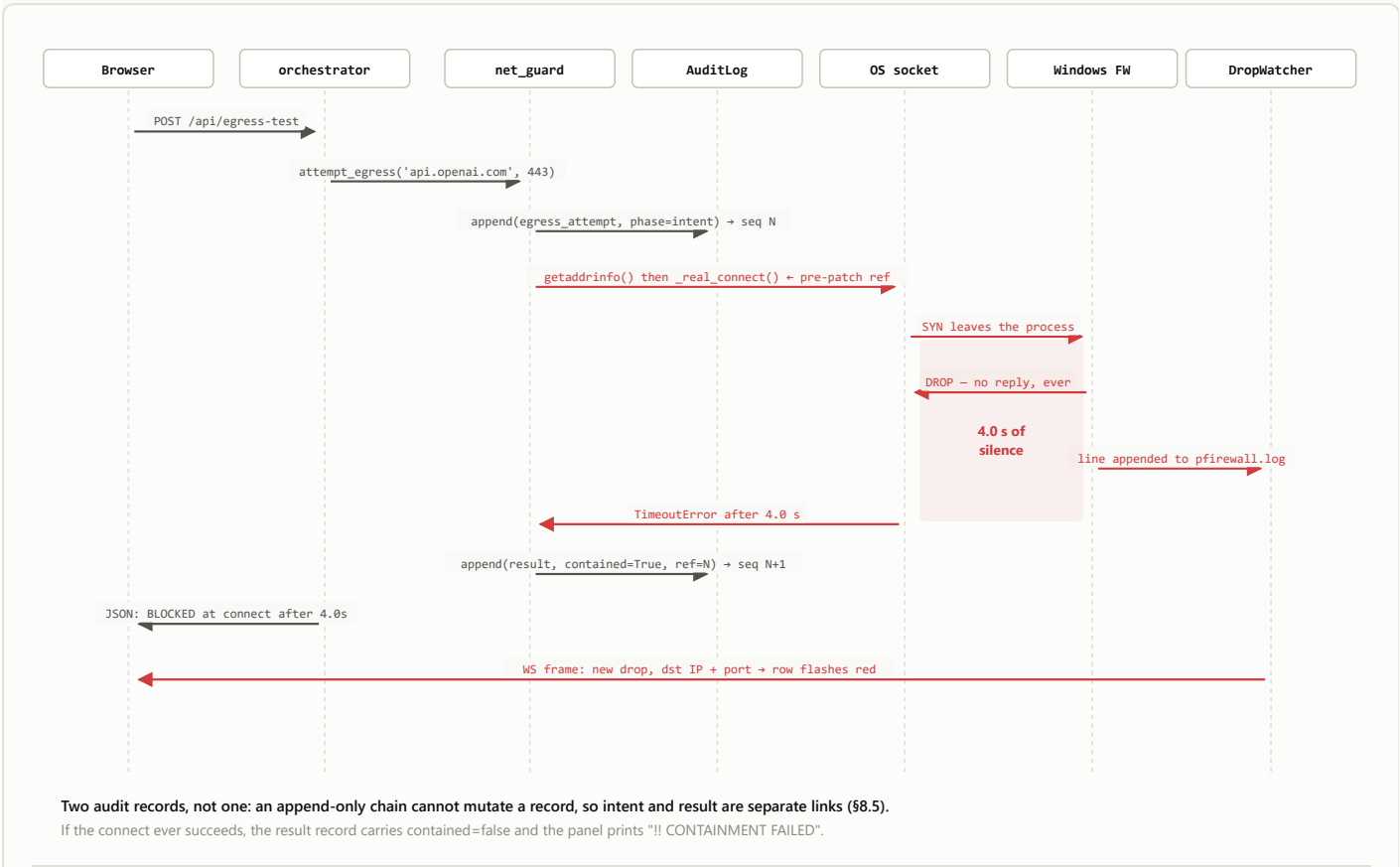


Figure 4.1 — Egress attempt, end to end. Red arrows are the path of the packet the firewall destroys. The critical design choice is at step 4: `attempt_egress()` calls a reference to `socket.connect` captured *before* `install_guard()` patched it, so our own Python cannot be what does the blocking. A guard that refuses the call in-process would prove only that we wrote an `if` statement.

PANEL READS

BLOCKED at connect
after 4.0s — TimeoutError

The elapsed time is measured, not hardcoded — a DNS-stage failure reports in milliseconds instead.

DROPS TABLE

a new row flashes red with the real destination IP and port

Parsed from the W3C-format Windows Firewall drop log, tailed incrementally.

AUDIT LOG

two linked records — intent, then result — with a red left border

The result record carries `ref: N` pointing back at the intent record's sequence number.

Why the drop log rather than drop counters

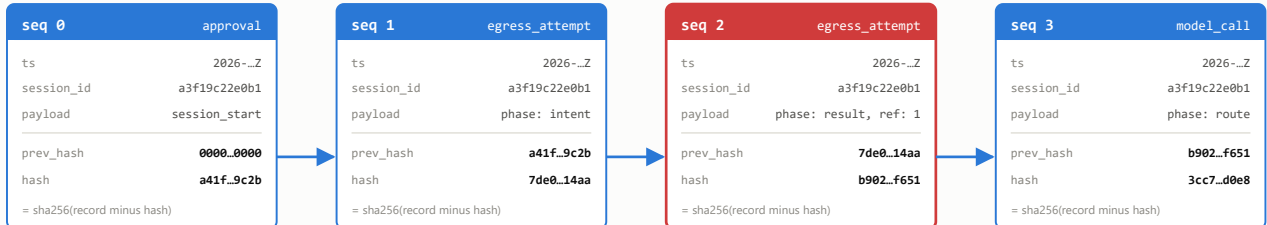
The charter was drafted against Linux and specified `nftables` drop-chain packet counters. The demo machine is Windows 11 with no general Linux userland — the only WSL distribution present is Docker Desktop's utility VM — so `nftables` has nowhere to run. Windows Firewall is the equivalent control, and it exposes no counter API. Tailing its drop log is the substitute, and it is strictly better for this demo: the panel shows the destination IP and port of the packet that died, not just an integer going up. This deviation is recorded in the charter rather than silently substituted, and it is worth saying out loud during the presentation — the same containment argument holds on the platform that PSU desktops actually run.

An append-only chain that cannot be quietly edited

Every model invocation, tool call, file read, file write, egress attempt and approval emits a record. The chain is framed as DPDP Act 2023 and CERT-In audit readiness in the presentation — for a PSU, this is a compliance feature rather than engineering decoration.

workspace/audit/audit.jsonl — append-only, one JSON object per line

The digest covers the whole record except `hash` itself — so seq, ts and kind are inside it too, not just the payload.



X payload edited by hand
hash no longer matches digest()

Three ways a record fails verification:

seq out of order · prev_hash ≠ predecessor's hash · hash ≠ recomputed digest

python -m core.audit verify workspace/audit/audit.jsonl

→ FAIL chain broken at seq 2 — and every record after it is orphaned.

Figure 5.1 — Chain structure and tamper detection. To demonstrate this live: edit one byte of any payload in the JSONL and re-run `audit verify`. It names the first bad sequence number. Because prev_hash is itself inside the digest, a forger would have to recompute every subsequent record to hide a single edit.

Two ambiguities in the charter, resolved in code

Digest scope. An earlier draft specified `sha256(prev_hash + canonical_json(payload))`. That leaves seq, ts and kind outside the digest — so a record's timestamp or kind could be rewritten without breaking the chain at all. The implementation hashes **the whole record minus hash**. Since prev_hash is a field of the record, the link to the predecessor survives the change.

Write-before versus update-after. The charter wants a record written before the action and the outcome recorded after it — but an append-only chain cannot mutate a record without invalidating everything downstream. Resolved by emitting **two** records: an intent record, then a result record whose payload carries `ref: <intent seq>`. Both are visible in Figure 4.1.

Record contract

```
core/audit.py — AuditRecord
seq      : int          monotonic from 0
ts       : str          ISO8601, UTC
session_id : str        12 hex chars
kind     : Literal[...] 7 values, closed set
payload  : dict
prev_hash : str          sha256 of predecessor
hash     : str          sha256(record - hash)
```

The seven kinds are `model_call · tool_call · file_read · file_write · egress_attempt · approval · error`. A closed literal set means an unrecognised kind fails validation at read time, so a hand-inserted record is caught even before the hash is checked.

Performance characteristics

One append is a single line write with no `fsync` — sub-millisecond, CPU only, no GPU implication. That is deliberate: it has to be cheap enough to call on *every* model call and tool call without distorting the latency figures quoted elsewhere in this report.

`tail()` currently reads the whole file, which is marked in the source as a deliberate shortcut with its ceiling named: fine to roughly 100k records, and a six-minute demo produces a few dozen. If the log ever outgrows RAM the fix is a reverse seek, and the comment says so.

The log resumes from an existing file on construction, so restarting the app continues the chain rather than starting a new one — which is what makes the verify step meaningful across a whole rehearsal rather than one process lifetime.

A deterministic router — no LLM decides which model runs

Determinism here is a contract, not a performance choice. The same prompt produces byte-identical output on every run, which is what makes an accuracy number meaningful and a routing decision auditable.

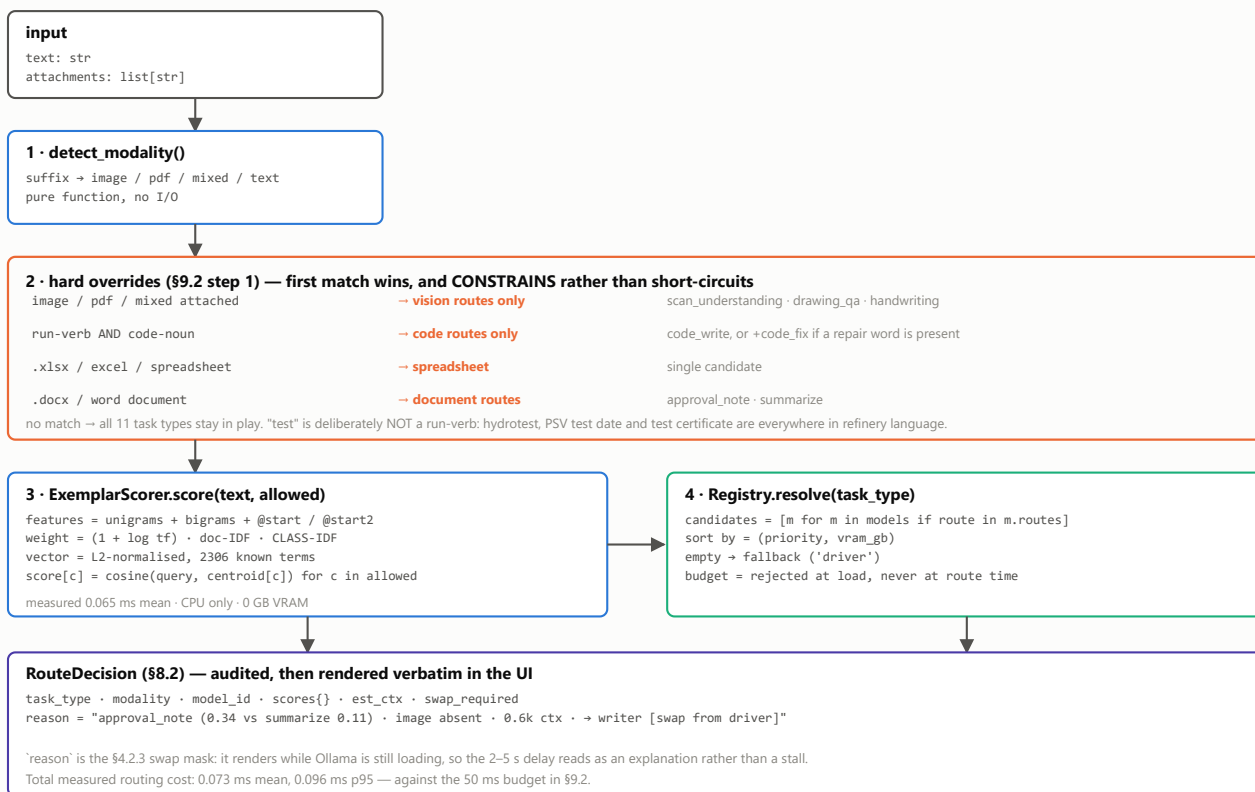


Figure 6.1 — The routing pipeline. Note step 2: hard overrides *constrain the candidate set* rather than short-circuiting to an answer. An attached drawing narrows the field to the three vision routes, and the scorer still has to choose between `drawing_qa` and `handwriting` — which keeps the decision explainable instead of arbitrary.

Class-IDF: the one modelling choice that earned its keep

Every task type in this corpus talks about thickness, inspection, reports and API codes. Those nouns are the *subject matter*, not the task. Standard document-IDF barely damps them, because they genuinely are frequent everywhere.

The scorer multiplies document-IDF by a **class-IDF** term: a word appearing in all 11 classes is damped to roughly 1.0, while one appearing in a single class — "handwritten", "draft", "summarise" — is amplified. Without it the router reads what the prompt is *about* and ignores what the user actually asked for.

The "test" trap

The charter's routing rule says literally: a request mentioning `run/execute/test` becomes a code task. In refinery language that rule is a trap — *hydrotest*, *PSV test date*, *test certificate*, *test pressure* are everywhere. The implementation drops "test" as a run-verb and instead requires a run-verb **and** a code noun to co-occur. Three regression tests pin this behaviour.

Latency against budget

680× under the §9.2 routing budget

50 ms budget (§9.2)

actual 0.073 ms — 0.15% of the bar, one hairline wide

50 ms

median 0.072 ms · p95 0.096 ms · max 0.149 ms
n = 176 prompts · full router incl. registry resolve

Deterministic: the same prompt yields byte-identical output every run. No LLM, no weights touched.

The one gate this build fails — and why

176 hand-labelled refinery prompts, 11 task types, 16 each. Every third prompt *within each class* is held out — a stratified 110/66 split. Hyperparameters were chosen by leave-one-out CV on the training split alone; the held-out set was measured once, at the end.

HELD-OUT ACCURACY

75.8%

50 of 66 correct

GATE

90%

shortfall of 14.2 points

CLASSES AT 100%

4

approval_note · calc · drawing_qa · plus a near-miss tier at 83%

WEAKEST CLASS

17%

qa — 1 of 6 correct

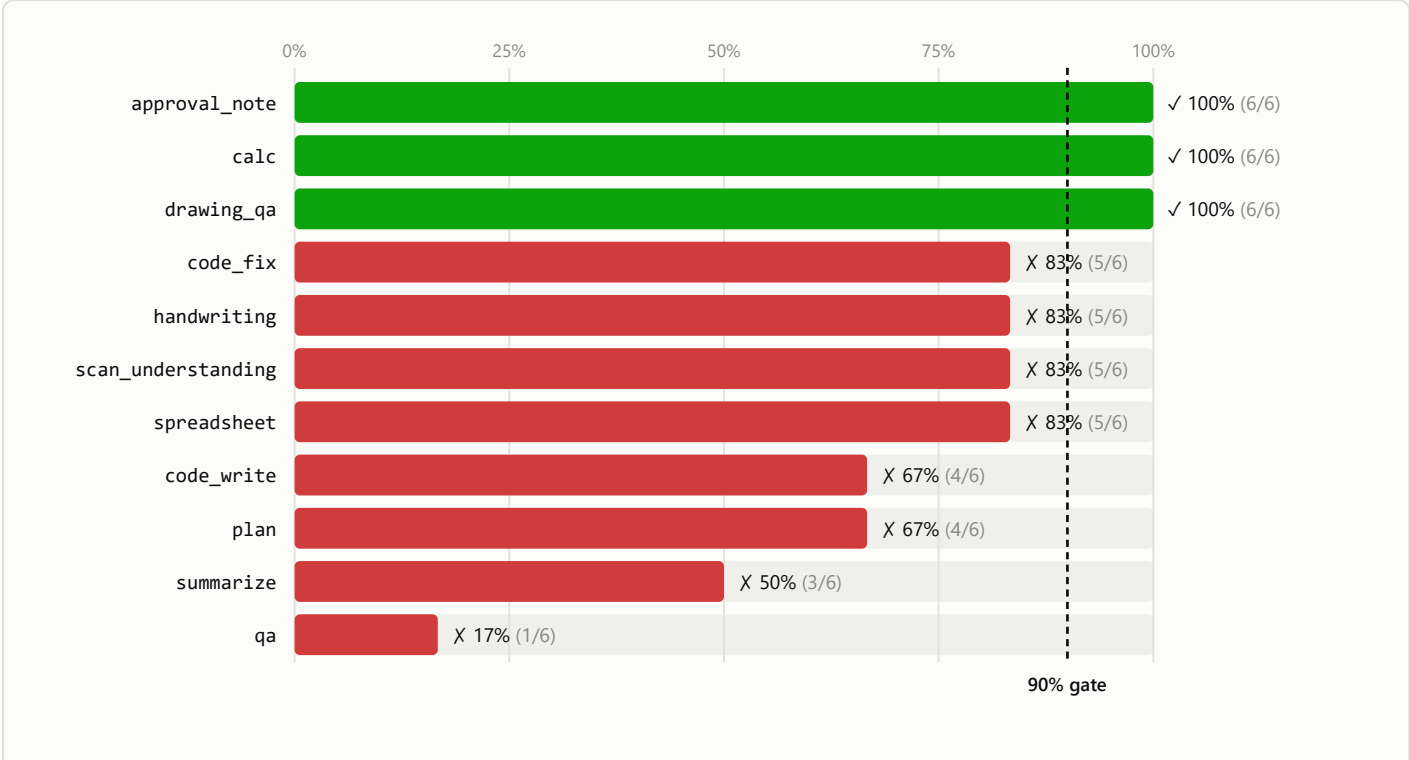


Figure 7.1 — Held-out accuracy by task type. Each class contributes exactly 6 held-out prompts, so per-class figures move in 16.7-point steps and should be read as coarse signal rather than precise rates. The shape is the finding: the shortfall is **concentrated in two classes**, not spread evenly across eleven.

The diagnosis: qa and summarize carry almost the entire gap. Together they account for 8 of the 16 held-out errors while representing only 18% of the held-out set. Both are the semantically *broadest* classes — a question can be about anything, and a summary request shares its whole vocabulary with the document it summarises. A lexical scorer has nothing distinctive to grip. The other nine classes average 87%, which is close enough to the gate that they are not the problem to solve.

Why this number is honest

Three properties make it quotable. The split is **stratified** — every third prompt *within each class* — so no class is over- or under-represented in the held-out set by accident. The scorer is rebuilt from the training prompts alone, so nothing is ever scored against itself. And the held-out set was touched exactly once, at the end: every choice of scorer variant and weighting was made by leave-one-out CV on the training split, because tuning against the held-out set would have turned the headline figure into decoration.

How the failure is recorded. The gate lives in `tests/test_router.py` as a strict `xfail`, with a second test pinning the floor at 70% so the number can only ratchet upward. Strictness is the point: when the encoder lands and accuracy clears 90%, the suite *fails* until someone deletes the `xfail` — so the gap can be neither silently forgotten nor silently fixed.

Where the router actually goes wrong

Aggregate accuracy hides structure. This matrix shows every one of the 66 held-out predictions at once, which turns a single disappointing percentage into a specific, addressable diagnosis.

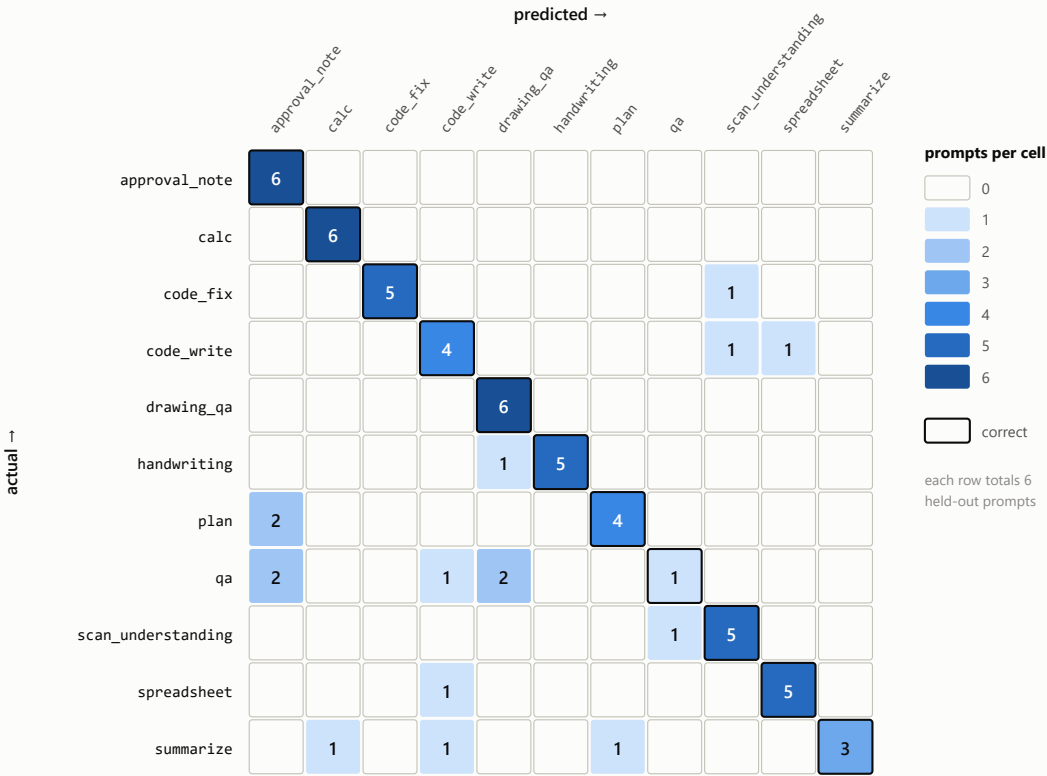


Figure 8.1 — Confusion matrix, 66 held-out prompts. Rows are the true label, columns the prediction; the outlined diagonal is correct, and every row totals 6.

The qa row is the pathology. Five of its six prompts are misrouted, and the mass is *scattered* — two to approval_note, two to drawing_qa, one to code_write — rather than concentrated in a single confusable neighbour. A pair of classes that trade prompts is a boundary problem, fixable with more exemplars. A class whose errors spray across five unrelated destinations has no lexical signature at all, and more exemplars will not create one.

The confusions that are almost reasonable. handwriting → drawing_qa and code_fix → scan_understanding are single off-diagonal cells between genuinely adjacent tasks, and in both cases the two routes resolve to the *same model* — so the user-visible outcome is unchanged. Counting them as failures is strict, and deliberately so: the gate measures the router, not the luck of the registry.

Why this is not a tuning problem

CLASS CENTROIDS (CHOSEN)

74.5%

LOO-CV on the training split

NEAREST EXEMPLAR

64.5%

10 points worse — too brittle at ~10 exemplars per class

INTENT-PHRASE BOOSTING

worse

at every setting tried

Three approaches within the lexical family land in the same band — a plateau, not a missed hyperparameter. The open decision is whether to reuse bge-m3 (ONNX int8, ~1.2 GB, CPU, already budgeted for retrieval) for both routing and retrieval, or stage a ~90 MB sentence encoder for routing alone. Either way the weights must be pre-staged on disk: fetching them at runtime is forbidden by the sovereignty invariant, so this is not a pip install away.

All 16 errors, in full

Every held-out prompt the router got wrong, with its text intact. Publishing the failures rather than the headline is the point: it is what lets a reviewer judge whether the gap is a boundary nuisance or a real defect.

HELD-OUT PROMPT	TRUE LABEL	ROUTED TO
Outline how we get from field UT readings to a signed recommendation for V-2301	plan	approval_note
Draw up a plan to digitise the last five years of inspection records for the HGU	plan	approval_note
What is the minimum required thickness for a 6 inch schedule 40 carbon steel line per API 570	qa	code_write
Who signs off a work permit for confined space entry in the FCC unit	qa	approval_note
Is a hydrotest required after a weld repair on a class 300 flange	qa	approval_note
Tell me what the vendor manual recommends as the overhaul interval for the crude charge pump	qa	drawing_qa
What are the acceptance criteria for pitting depth on a tank floor plate	qa	drawing_qa
Write a brief overview of what happened in the SRU during the last shutdown	summarize	code_write
Give me the highlights of this API 653 tank inspection report	summarize	plan
Sum up what the last five years of thickness surveys tell us about this circuit	summarize	calc
Write code to batch rename scanned report files by equipment tag and date	code_write	scan_understanding
Create a script to compare two revisions of the equipment list and show what changed	code_write	spreadsheet
This script worked last month and now throws an encoding error on the scanned filenames	code_fix	scan_understanding
What does the attached scanned work permit say about the validity period	scan_understanding	qa
Read the scribbled note the inspector left on the drawing	handwriting	drawing_qa
Build me a tracker with one row per finding and a status column	spreadsheet	code_write

Reading the errors. Several are defensible rather than absurd — "Outline how we get from field UT readings to a signed recommendation" genuinely sits between plan and approval_note, and both would route to a reasonable model. Others are clearly wrong in a way a dense encoder would fix: "What is the minimum required thickness for a 6 inch schedule 40 carbon steel line per API 570" routing to code_write is a lexical accident, driven by shared numeric and specification vocabulary rather than by any sense of what was asked. "Sum up what the last five years of thickness surveys tell us" landing on calc is the same failure in miniature: the scorer reads the nouns and misses the verb.

5.2 GB, not 6 — and one model resident at a time

The demo machine is an HP Omen laptop with an RTX 4050 (6 GB GDDR6) and 64 GB of system RAM. That GPU is the entire design constraint; every component in this report states its VRAM cost, and most of them state zero.



Figure 10.1 — Budget derivation. Designing against 6.0 GB is the standard way to run out of memory mid-demo. The display, compositor and driver take roughly 0.5 GB before any model loads, and CUDA context plus allocator fragmentation takes another 0.3 GB.

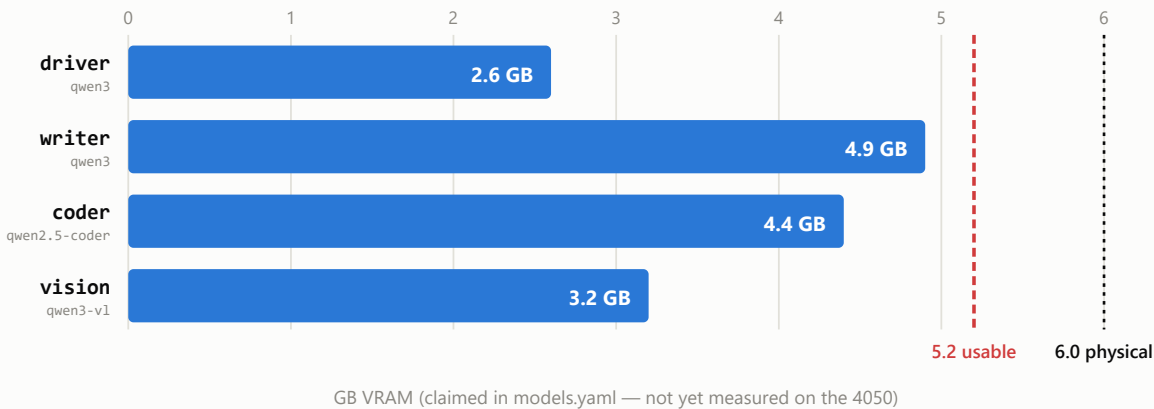


Figure 10.2 — Claimed VRAM per registered model. These are *claims read from config*, not measurements — quantisation sizes drift between releases, and the charter requires re-verification with `ollama ps` and `nvidia-smi` on the actual card before any of them is trusted. The registry enforces the claim at load time: a model declaring more than the profile budget is rejected before it can ever be routed to.

The four models sum to 15.1 GB — and that is fine. The GPU is single-tenant for LLMs by policy (`OLLAMA_MAX_LOADED_MODELS=1`), so the number that must fit under 5.2 GB is the *largest single* model, not the total. Today that is *writer* at a claimed 4.9 GB, leaving 0.3 GB of headroom — the tightest fit in the roster and the one most likely to move when it is finally measured.

What runs on CPU instead

Embeddings, reranking and classical OCR are all specified to run on CPU. 64 GB of system RAM is the asset on this machine; GPU time is reserved for generation only. That policy is also what makes the dense-encoder fix on page 07 affordable — the router’s encoder would not compete with the LLM for VRAM at all.

Registry contract — enforced at load, not at route time

FIELD	RULE
id	unique; duplicates rejected
backend	ollama llamacpp
device	gpu cpu — only gpu is budget-checked
vram_gb	must be ≤ profile budget (5.2)
max_ctx	hard ceiling; retrieval truncates to it
routes	task types this model declares
priority	tie-break, then lower vram_gb wins
requires_tools	surfaced in the decision rationale

A route with no declaring model falls back to *driver*, and the fallback is itself validated — a registry whose fallback id was rejected fails to load rather than serving a dangling reference.

Adding a model is editing one file

This is a graded requirement, so it is protected by tests rather than by discipline: no model name appears anywhere in `core/`. All three outcomes below were verified with zero code change and zero restart.

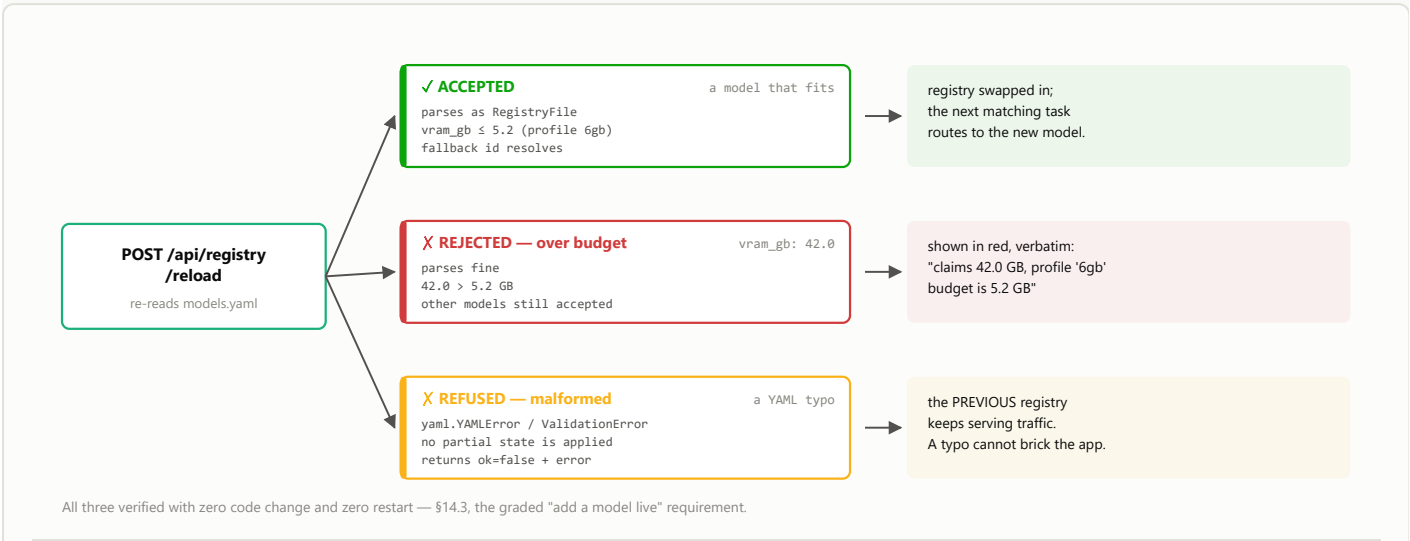


Figure 11.1 — Reload outcomes. The third path matters more than it looks: a YAML typo pasted on stage cannot brick the application. Validation happens against a fully parsed candidate registry, and the live one is only swapped in after every check has passed — so a partial or malformed edit is rejected as a whole.

Acceptance gates — measured status

GATE	TARGET	ACTUAL	STATUS
pytest tests/	green before any merge	36 passed, 1 xfailed (0.66 s)	✓ MEASURED PASS
Router accuracy, held out	≥ 90%	75.8% – 50/66 correct	✗ MEASURED FAIL
Routing latency	< 50 ms (\$9.2)	0.073 ms mean, 0.096 ms p95	✓ MEASURED PASS
Corpus size	≥ 100 labelled prompts	176 prompts, 11 classes × 16	✓ MEASURED PASS
mypy --strict on core/ + sovereignty/	clean	clean	✓ MEASURED PASS
Egress packets delivered during a run	0, drops counted and shown	0 by construction; needs a stage run to quote	⦿ NOT YET RUN
sovereignty/verify.ps1	exits 0	needs an elevated shell – rehearse it	⦿ NOT YET RUN
Peak VRAM during demo	< 5.2 GB, logged	0 GB today – no model is loaded until leg 4	○ NOT YET APPLICABLE
Golden demo path end to end	< 6 min wall clock	legs 4–8 not built	○ NOT YET APPLICABLE
Generated .docx opens in MS Word	manual, every build	leg 4	○ NOT YET APPLICABLE
Sandbox pass rate, 10 coding tasks	≥ 70% first run	leg 5	○ NOT YET APPLICABLE

The distinction between **MEASURED FAIL** and **NOT YET APPLICABLE** is deliberate. One gate is failing and known; the rest are simply not reachable until the legs that exercise them are built. Presenting an unbuilt leg as a passing gate would be the easiest way to lose credibility in questioning.

Three legs deep, not six shallow

3 of 8 legs built · first VRAM measurement arrives with leg 4

1	Sovereignty panel + red button + audit log	✓ built
2	Registry + deterministic router + 2 models	✓ built
3	Live model addition (hot reload)	✓ built
4	Agent loop, 3 tools → .docx approval note	► next
5	Coding task in the --network none sandbox	○ planned
6	Multimodal: handwriting / drawing title block	○ planned
7	Ingest + hybrid retrieval over 10–15 docs	○ planned
8	Cable pull	○ planned

Figure 12.1 — The golden demo path. Build order is fixed by the charter, and anything that does not serve this path is deferred. Leg 4 is the first that actually loads a model, which makes it the first real measurement of VRAM against the 5.2 GB ceiling — every VRAM figure in this report is a config claim until that leg exists.

Immediate next steps, in order

1 · Settle the encoder question. Router accuracy is the only currently failing gate, and it blocks quoting a credible number on a slide. Choose between reusing bge-m3 and staging a small routing-only encoder; stage the weights on disk either way.

2 · Build leg 4 — the agent loop. Three tools only (fs_read, py_sandbox, doc_write): scanned inspection report → extracted findings → drafted approval note → .docx. Hard caps of 8 steps and 20k cumulative tokens, with every step audited and streamed to the UI as a visible plan trace.

3 · Rehearse the launch sequence. The drop-log monitor needs read access to an Administrator-only file. Decide between running elevated and relaxing that one ACL *before* rehearsal — and test whether the laptop thermally throttles after 20 minutes of sustained inference. A throttle mid-demo is a silent killer.

Scaling: same code, different registry

PROFILE	BUDGET	CONSEQUENCE
6gb	5.2 GB	RTX 4050 Laptop — the demo machine. One LLM resident; swaps are masked by the routing rationale.
16gb	14.5 GB	RTX 4080 / A4000 class. Headroom for a 20B-class driver plus a resident vision model, so most swaps disappear.
120gb	110 GB	Server class (H100, or 2× A6000). A 120B-class model stays resident and the swap-masking UI becomes dead code.

The 16 GB and 120 GB profiles are a documented scaling story, not a measured claim — they ship as config and are labelled as unimplemented in their own comments. What they demonstrate is that the answer to "deployable on a mid-range GPU, scalable to your real server" is a registry swap rather than a rewrite: the router, the audit chain, the containment layers and the UI are all indifferent to which profile is loaded.

Platform deviations, recorded not hidden

CHARTER (LINUX)	THIS REPOSITORY (WINDOWS 11)
nftables OUTPUT DROP	Set-NetFirewallProfile -DefaultOutboundAction Block
allow lo + local subnet	outbound allow rule for LocalSubnet; loopback is never filtered
drop-chain counters	pfirewall.log — richer: names the dead packet's IP and port
verify.sh	verify.ps1 — same five assertions
Docker sandbox	unchanged — Docker 29.6.1 is present, no substitution needed